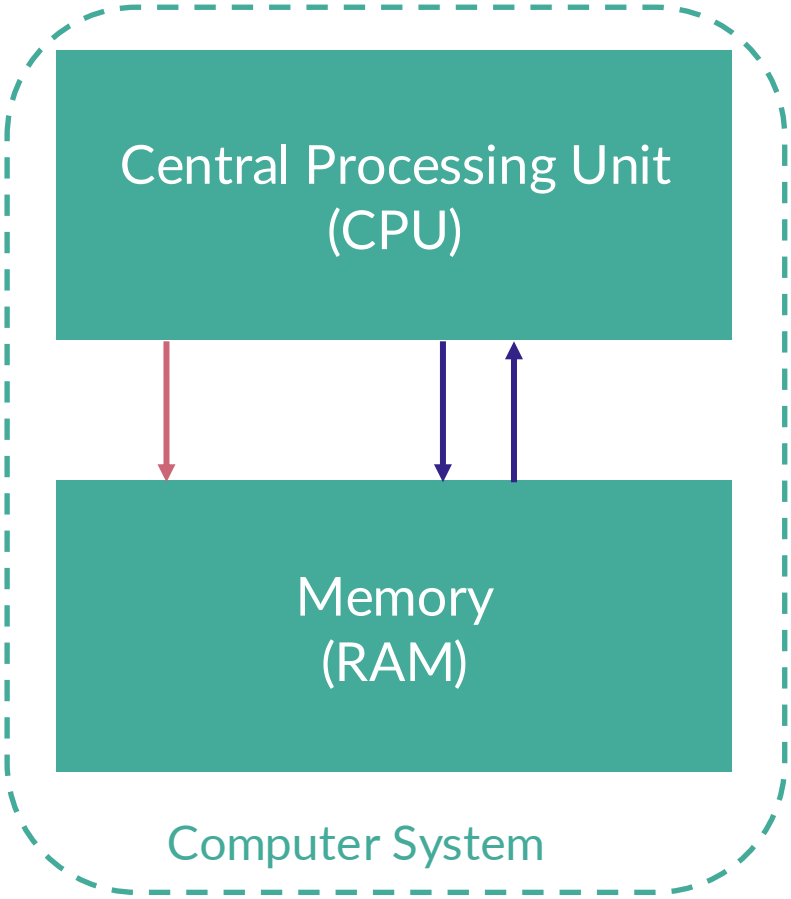
CS 255: Computer Architecture and Machine Level Programming

Prof. Martha Barker

Caches

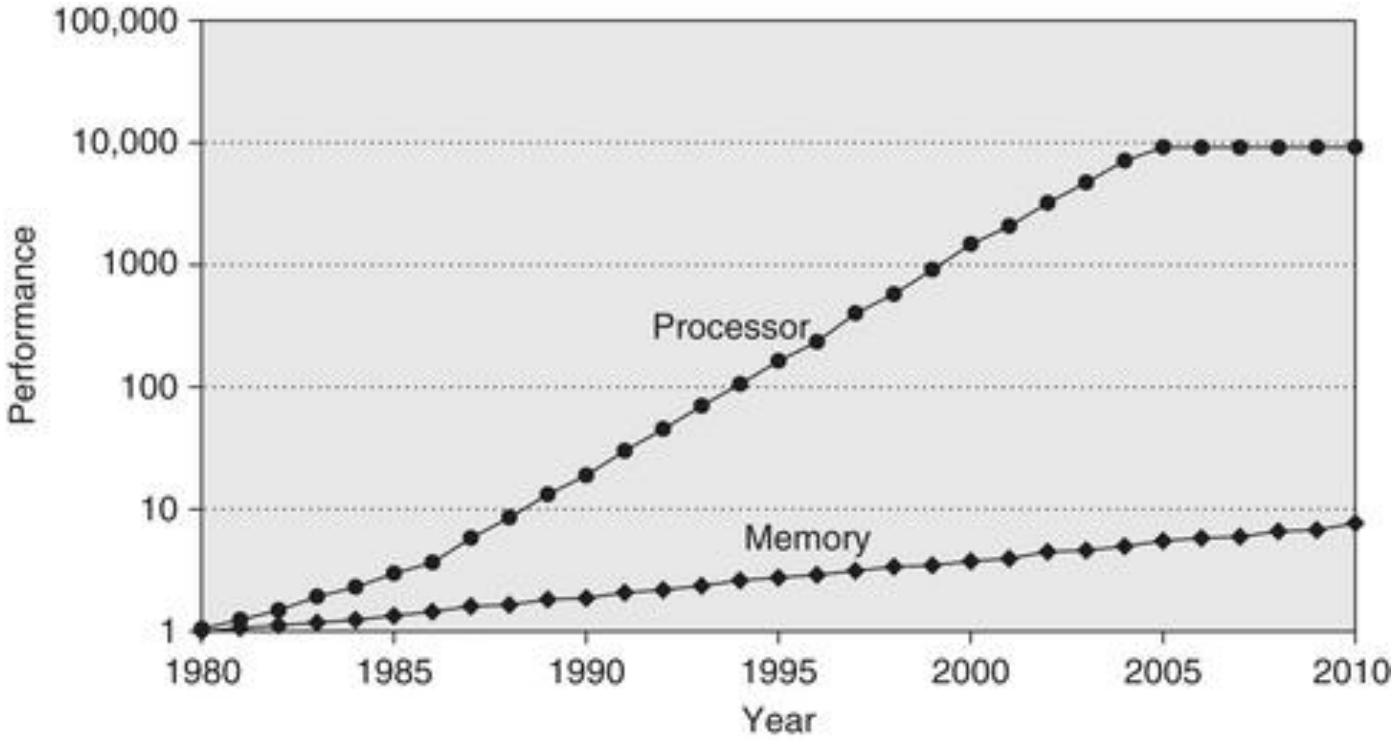
Computer System Performance

System performance depends on the slowest component.



Processor-Memory Performance Gap

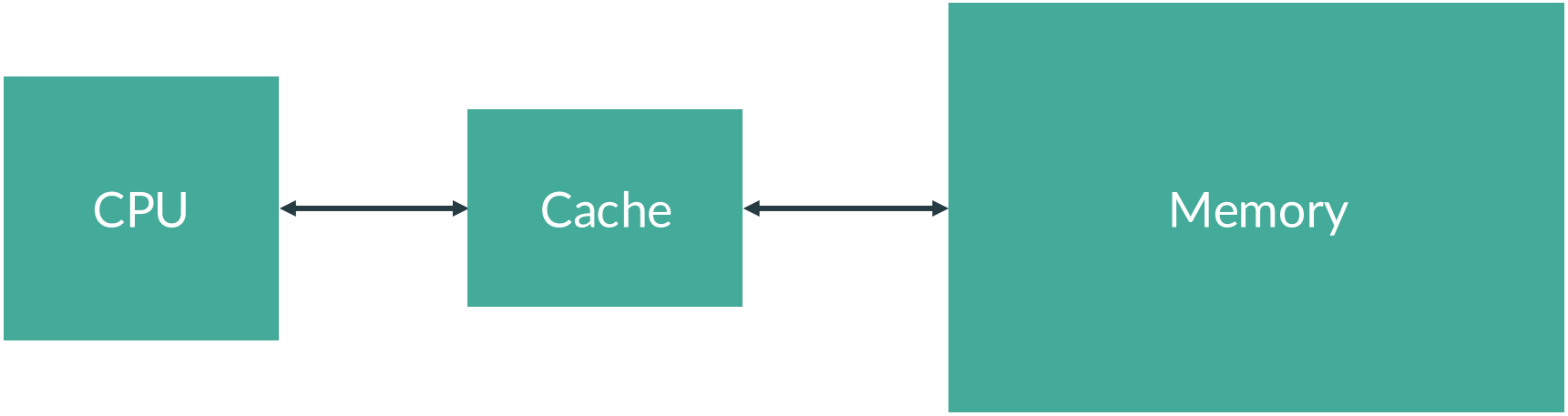
Memory speeds have not kept up with processor speeds.



Hennessy and Patterson. *Computer Architecture: A Quantitative Approach*. 5th ed.

Caches Bridge The Gap

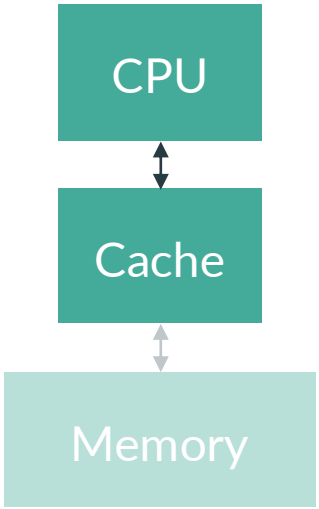
A cache is a small memory with fast access times that stores a subset of an address space.



Cache Hits and Misses

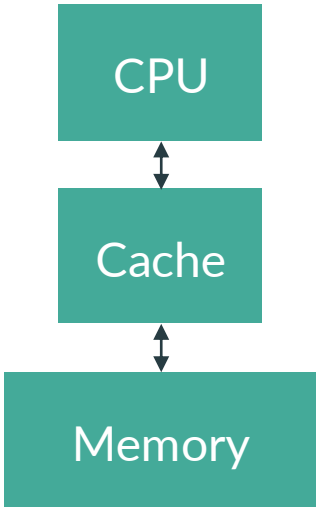
Hit: data found in cache

- CPU requests data
- data found in cache
- data returned to CPU



Miss: data not found in cache

- processor requests data
- data not found in cache
- data is fetched from memory and stored in cache
- data returned to CPU



Caches Exploit Locality

Most programs use data that has been used recently or is close to data that has been used recently.

Temporal locality:

- data that has been used recently is likely to be used again soon

Spatial locality:

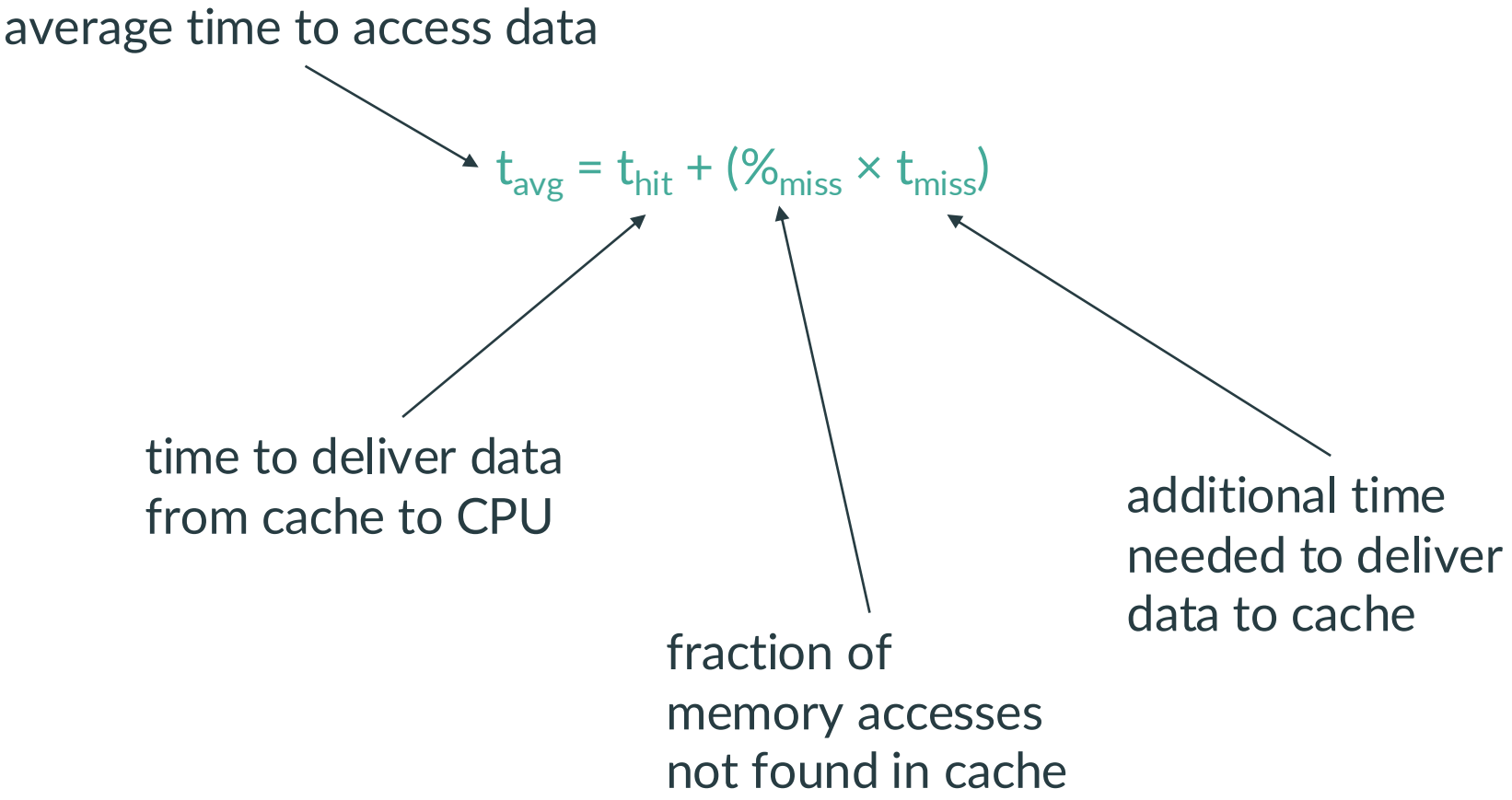
- data stored close to data that has been used recently is likely to be used soon

Locality Example

Where is there locality in this program?

```
int sumArray(int arr[], int n) {
  int sum = 0;
  for (int i = 0; i < n; i++) {
    sum += arr[i];
  }
  return sum;
}
```

Memory System Performance



Example: Memory Performance

Suppose a program does 2000 loads and stores and 1250 of those data values are found in the cache.

What are the hit and miss rates for the cache?

Example: Memory Performance

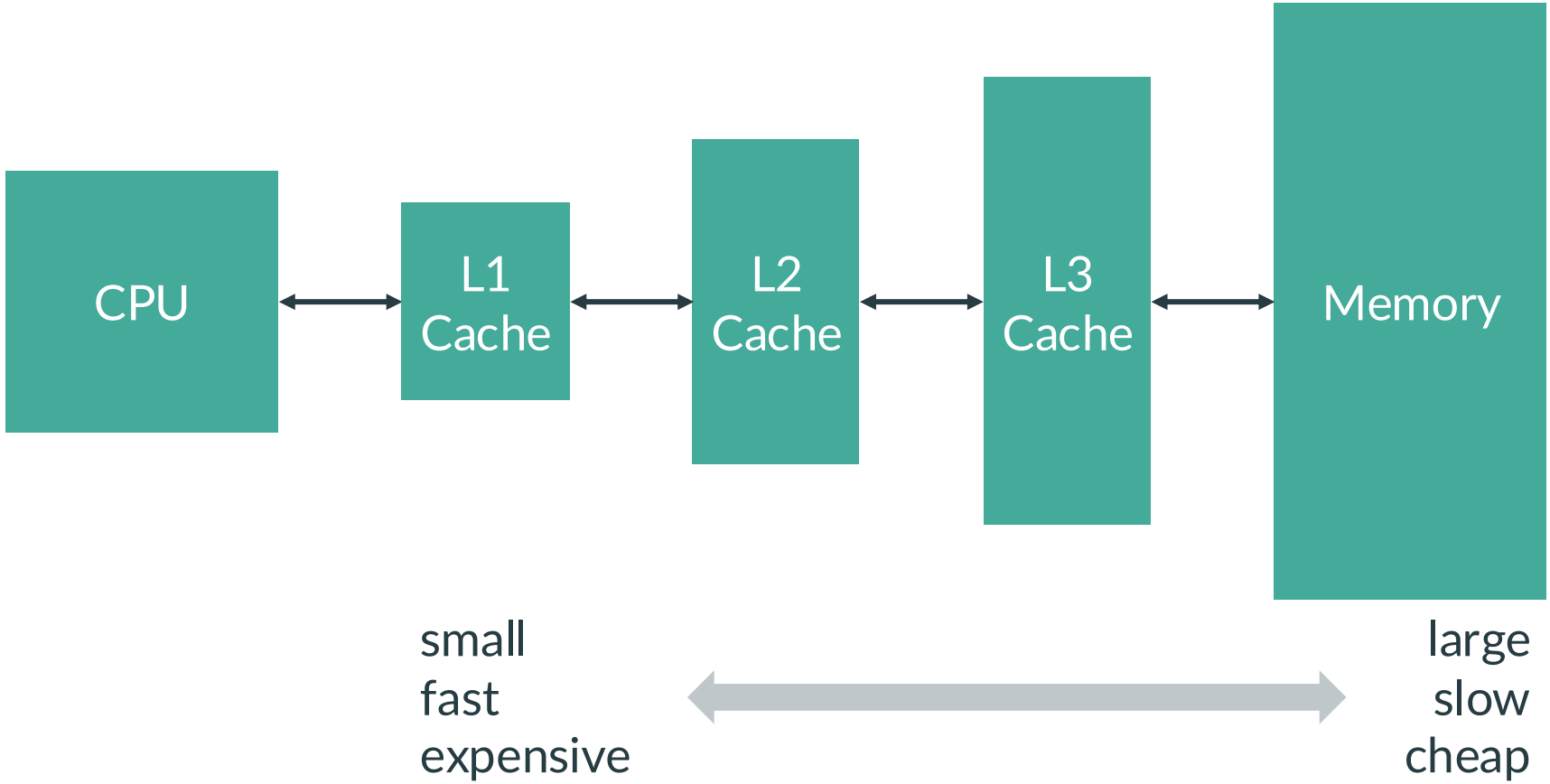
Suppose a program does 2000 loads and stores and 1250 of those data values are found in the cache.

What are the hit and miss rates for the cache?

If the access time of the cache is 1 cycle and the access time of memory is 100 cycles, what is the average memory access time of the program?

Memory Hierarchy

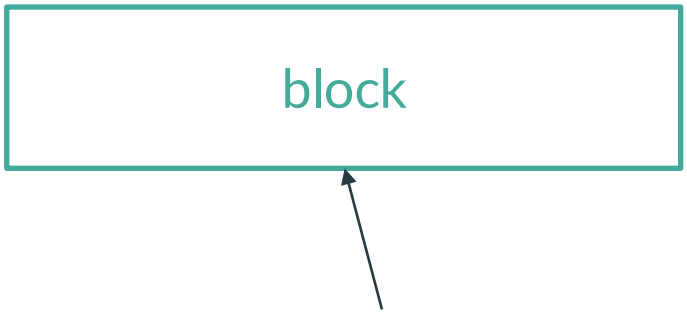
Use a hierarchy of memory components to improve performance.



Cache Organization

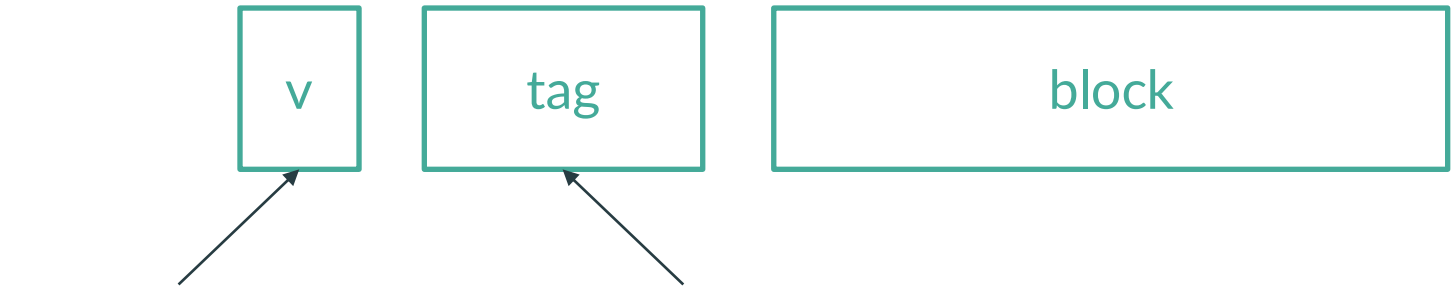
Data Is Stored in Blocks

A block is a fixed size chunk of data that is stored in a cache.



block sizes are typically larger than a machine word to exploit spatial locality

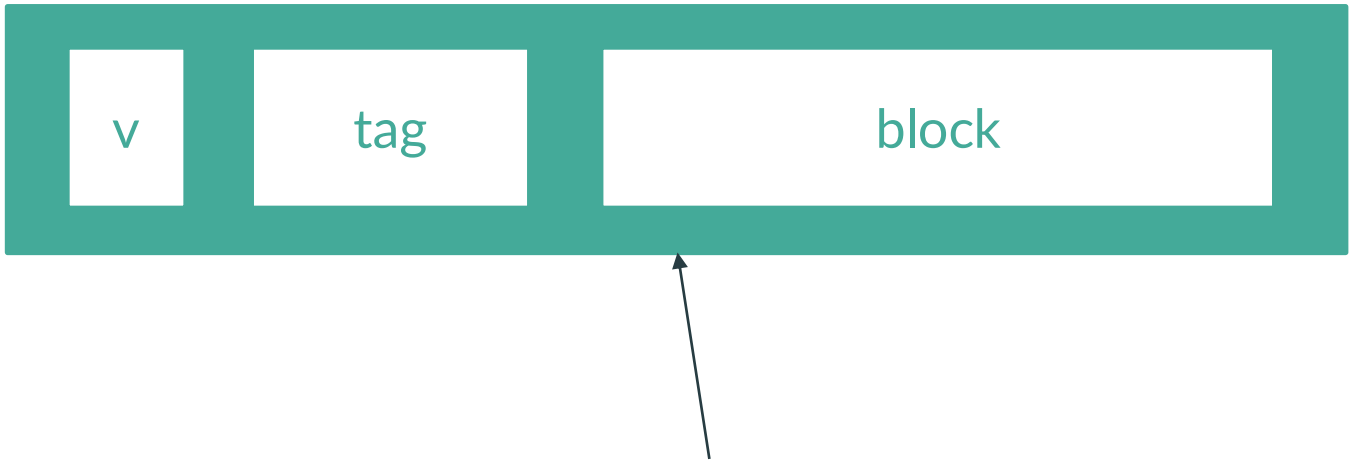
Blocks Need Metadata



a valid bit indicates if the block contains valid data

tag bits specify which data is stored in the block

A Cache Line Stores One Block



a block and its metadata are called a cache line

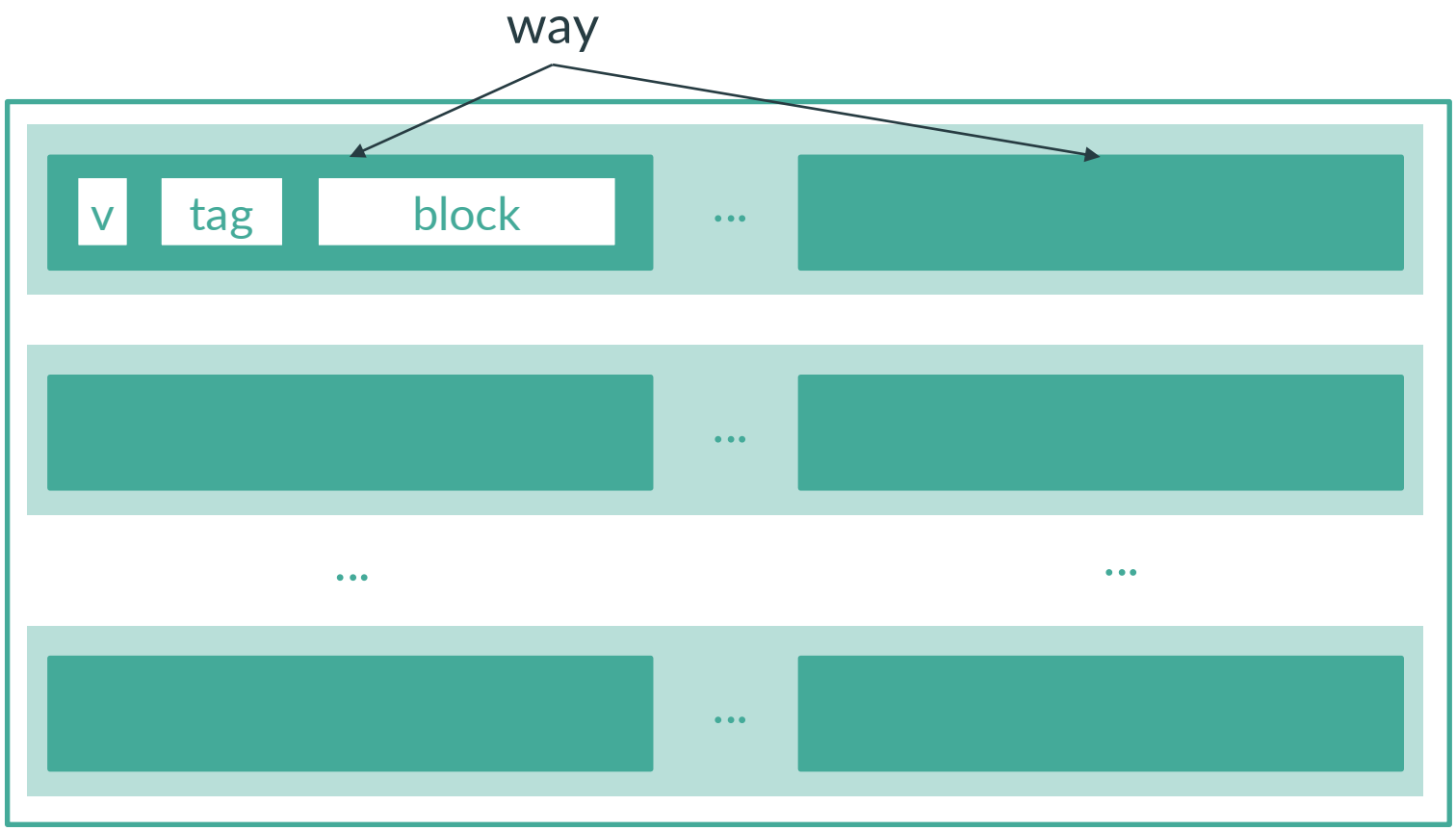
A Cache is Organized into Sets

Each memory address maps to exactly one set.



Each Set Contains 1 or More Blocks

Each set can store a block in one more than one way within it.



Cache Capacity

The capacity of a cache is the number of data bytes it can store.

capacity = block size × number of sets × number of ways per set

*tag and valid bits are not counted as part of capacity

Associativity

Associativity is the degree of flexibility in where data can be stored.

direct-mapped

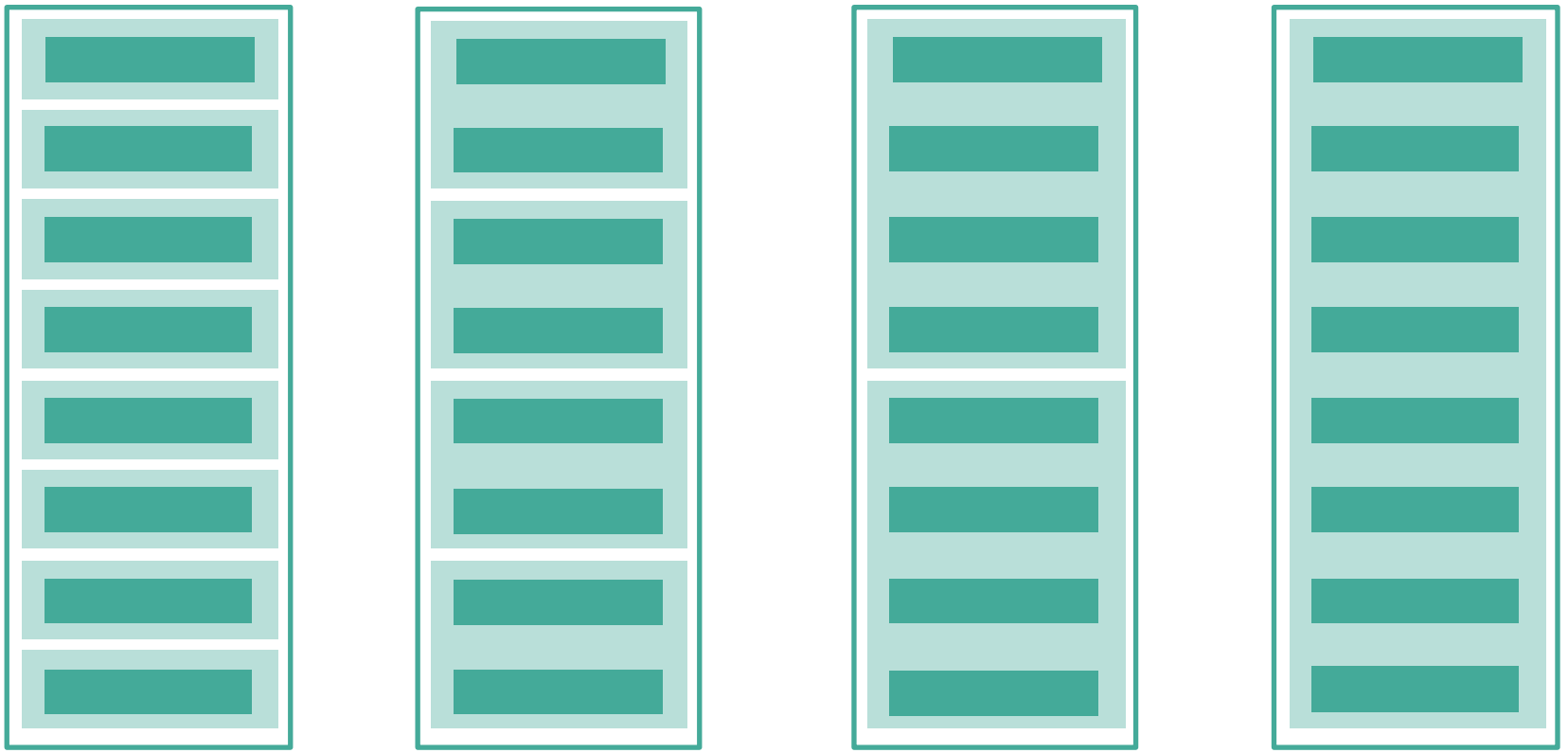
fully associative

1-way

2-way

4-way

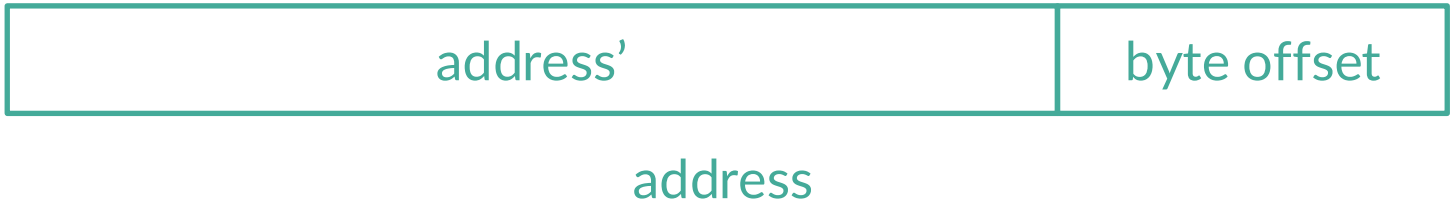
8-way



Mapping Addresses to Bytes

A block can contain multiple machine words, so how do you know which one to transfer to the CPU?

byte offset = address % block size
byte offset = low-order $\log_2$(block size) bits of address



Mapping Addresses to Sets

Each address maps to exactly one set, so how do you know which one?

set index = address' % number of sets
set index = low-order $\log_2$(number of sets) bits of address'



Mapping Addresses to Tags

Many different possible addresses map to the same set, so how do we know which one is currently being stored?

all remaining address bits are the tag
(which corresponds to the number of blocks that could be in the set)



Example: Cache Behavior

Suppose you have a 32-byte direct-mapped cache with 8 byte blocks and 8 bit addresses.

Trace the memory accesses of the following program.

```
int sumArray(int a[], int b[], int n) {
  int sum = 0;
  for (int i = 0; i < n; i++) {
    sum += a[0] + b[i];
  }
  return sum;
}

sumArray(a, b, 4); // &a[0] = 0x00, &b[0] = 0x18
```

Types of Cache Misses

Compulsory Miss

- first time block is accessed

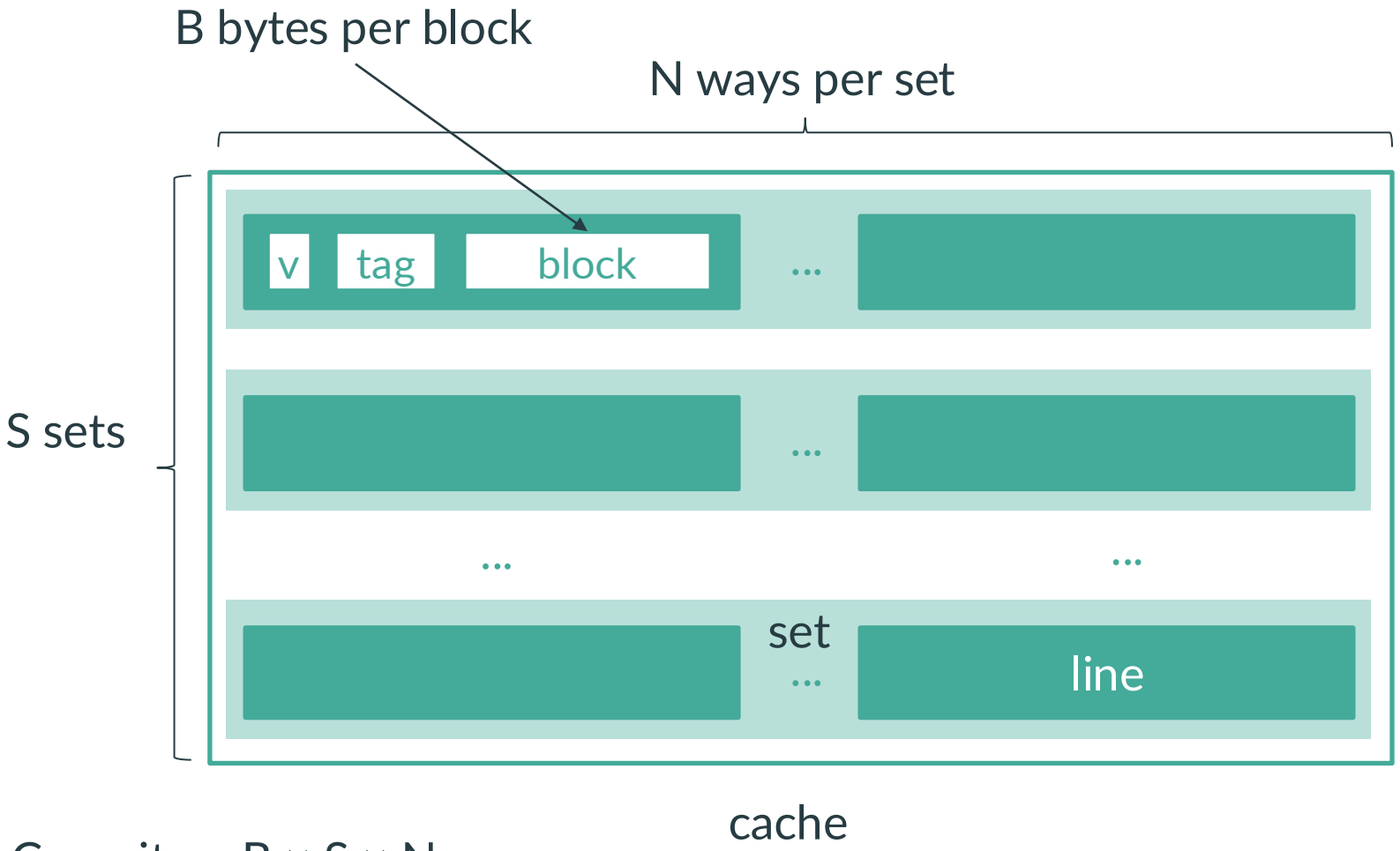
Capacity Miss

- the set of active cache blocks is larger than the cache

Conflict Miss

- cache is large enough, but multiple blocks map the the same location

Cache Organization Recap



Capacity = B × S × N